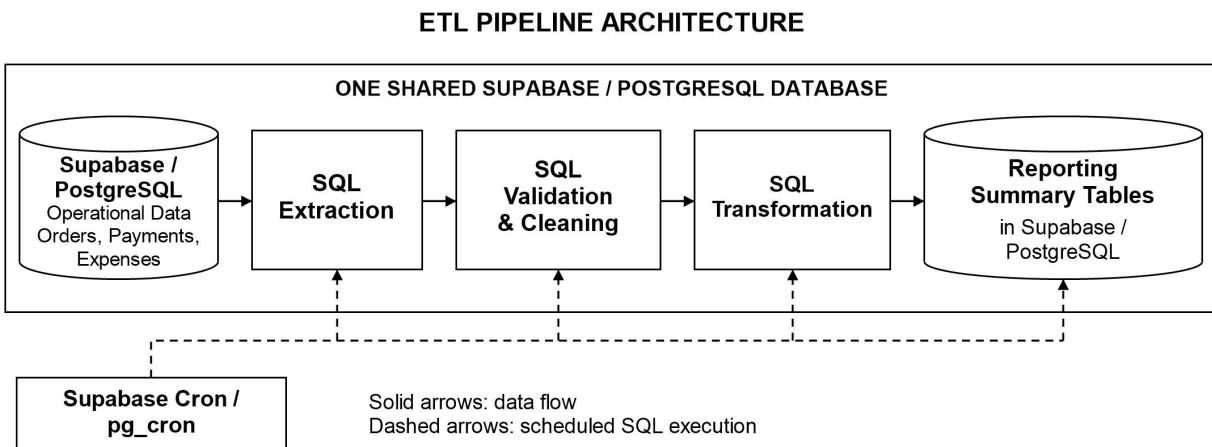


## 1. Pipeline Overview

The proposed I&C Laundry Daily Branch and Service Reporting ETL Pipeline will extract operational records from the shared Supabase PostgreSQL database used by the Main, Calzada, and Nasugbu branches. Its purpose is to prepare consistent information for sales monitoring, payment collection reporting, service-demand analysis, branch comparisons, and daily customer activity. The design retains the previously selected stack: PostgreSQL / Supabase, Supabase Cron / pg\_cron, SQL, and Reporting Summary Tables.

The pipeline will validate and standardize committed records, calculate daily measures, and load two reporting tables in the same database. Order value will be reported on the order creation date, cash received on the payment collection date, and expenses on the recorded expense date. These measures will support historical dashboards and reports and can provide inputs for forecasting; forecast generation itself is outside this pipeline. Source definitions follow the database schema supplied by the project owner. The scheduled ETL function, staging tables, reporting tables, and application integration remain proposed, not confirmed as deployed.

## 2. Pipeline Architecture



All processing and storage remain in PostgreSQL. Solid arrows show data movement between stages; dashed arrows show scheduled orchestration of the SQL refresh. The reporting tables would be read through the application's authorized reporting layer.

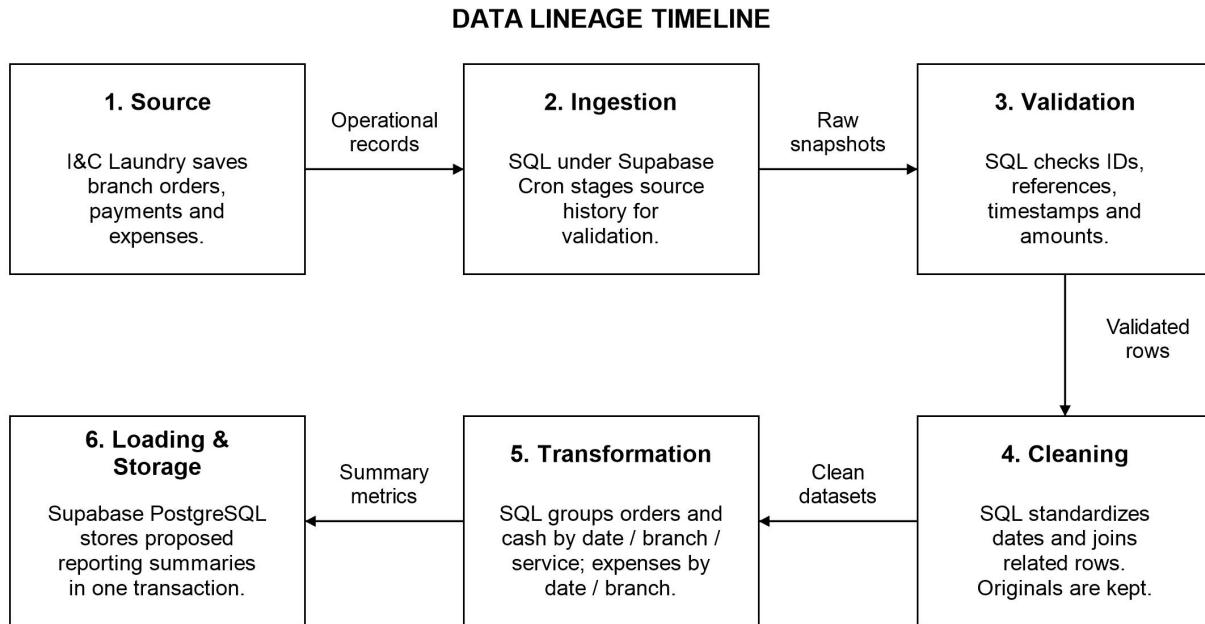
### 3. Pipeline Metadata

| Metadata                         | Description                                                                                                                                                                                                                                |
|----------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Pipeline Name</b>             | I&C Laundry Daily Branch and Service Reporting ETL Pipeline                                                                                                                                                                                |
| <b>Purpose</b>                   | Extract, validate, clean, transform, and load daily measures for branch operations, service demand, order value, received payments, expenses, and customer activity.                                                                       |
| <b>Technologies / Tools Used</b> | PostgreSQL / Supabase; Supabase Cron / pg_cron; SQL; Reporting Summary Tables.                                                                                                                                                             |
| <b>Data Source</b>               | Six tables from the supplied schema: public.orders, public.payments, and public.expenses, with public.branches, public.service_types, and public.customers as references. Use branch IDs for grouping and branches.branch_name for labels. |
| <b>Destination / Storage</b>     | Proposed analytics.daily_branch_summary and analytics.daily_service_summary tables in the existing Supabase PostgreSQL database.                                                                                                           |
| <b>Ingestion Tool</b>            | PostgreSQL SQL SELECT statements copy source records into transaction-local staging tables. No external ingestion server is introduced.                                                                                                    |
| <b>Transformation Tool</b>       | SQL performs validation, timezone conversion, joins, filtering, independent aggregation, and reconciliation before publishing results.                                                                                                     |
| <b>Orchestration Tool</b>        | Supabase Cron / pg_cron invokes the proposed analytics.refresh_daily_reports() function. Job name: ic_laundry_daily_reports.                                                                                                               |
| <b>Data Storage</b>              | Operational records stay in public tables. Proposed reporting tables are stored in analytics. Temporary staging data exists only during the refresh transaction.                                                                           |
| <b>Schedule</b>                  | Daily at 01:00 Asia/Manila (UTC+08:00). With the scheduler configured for UTC, the cron expression is 0 17 * * *. Process business dates earlier than the local date at run start.                                                         |

| <b>Metadata</b>                         | <b>Description</b>                                                                                                                                                                                                                                                                                                                               |
|-----------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Dependency</b>                       | Source tables matching the supplied schema; valid branch and order references; missing required reporting dates or branch assignments resolved. Verify pg_cron, the proposed function and targets, and authorized source-read/target-write permissions before deployment.                                                                        |
| <b>Configurations</b>                   | PHP currency; Asia/Manila business dates; exact NUMERIC arithmetic; one consistent source snapshot and run timestamp; transaction-scoped advisory lock to prevent overlapping runs. Rebuild completed-day history to include corrections and late collections. A failed validation rolls back publication; inspect job history before rerunning. |
| <b>Connections Between Technologies</b> | Supabase Cron invokes SQL in PostgreSQL. SQL reads the shared operational tables, stages and groups records, then writes both reporting tables in one transaction. Authorized application queries consume the published summaries.                                                                                                               |
| <b>Pipeline Steps</b>                   | <ol style="list-style-type: none"> <li>1. Source operational records.</li> <li>2. Ingest a consistent snapshot.</li> <li>3. Validate keys, dates, and amounts.</li> <li>4. Clean and standardize staged data.</li> <li>5. Transform into daily metrics.</li> <li>6. Load and store reconciled summaries.</li> </ol>                              |

## 4. Data Lineage

The diagram follows one scheduled ETL run. Its six stages describe the sequence of transformations, not their execution duration. Source records are preserved; cleaning takes place in the staged copy.



| Stage           | Input                                                        | Process                                                                                                                                                                            | Output                                          |
|-----------------|--------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------|
| Source          | I&C Laundry operations across three branches.                | The application records orders, payment-ledger entries, expenses, and reference data in the shared Supabase PostgreSQL database.                                                   | Committed operational records.                  |
| Ingestion       | Source tables and related branch, service, and customer IDs. | Cron invokes SQL. Capture one transaction snapshot and run timestamp; copy relevant history into temporary stages. Retain invalid or missing dates for validation.                 | Raw staged records at their original row grain. |
| Data Validation | Staged orders, payments, and expenses.                       | Check IDs, dates, branches, order links, allowed statuses, and amounts. Nullable source branches and order dates must resolve before publishing. Invalid records stop the refresh. | Validated source records.                       |

| Stage                      | Input                             | Process                                                                                                                                                                                          | Output                                   |
|----------------------------|-----------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------|
| <b>Cleaning</b>            | Validated staged records.         | Derive local dates from orders.created_at and payments.paid_at; retain expenses.expense_date. Join branches by ID, use branch_name for labels, and group null services as Unspecified.           | Consistently dated and related datasets. |
| <b>Transformation</b>      | Clean datasets through yesterday. | Aggregate orders, cash collections, and expenses independently by their own dates and branch. Group orders and cash by service separately. Merge aggregate keys without multiplying detail rows. | Daily branch and service metrics.        |
| <b>Loading and Storage</b> | Reconciled aggregate datasets.    | Check totals against each source and across summaries. Replace both target tables atomically and set refreshed_at. Roll back on failure; repeat runs do not append duplicates.                   | Published Reporting Summary Tables.      |

## 5. Schema Metadata

Source definitions follow the owner-supplied 21-table schema, not a direct database inspection. This table lists the six sources and selected columns used by the ETL, followed by two proposed targets; it is not a full application data dictionary. PK means primary key; FK means foreign key. “NULL” marks nullable columns; other listed columns are NOT NULL. Source NUMERIC fields have no declared precision or scale.

| Database Table                               | Columns and Data Types                                                                                                                                                                                 | Primary Key | Foreign Keys and Relationships                                                                                                              |
|----------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------|---------------------------------------------------------------------------------------------------------------------------------------------|
| public.branches<br>(existing source)         | id — UUID<br>branch_name — TEXT<br>name — TEXT (NULL)<br>code — TEXT (NULL)                                                                                                                            | id          | One branch has many orders, payments, expenses, and reporting rows. branch_name supplies the label; name and code are optional.             |
| public.customers<br>(existing reference)     | id — UUID                                                                                                                                                                                              | id          | Referenced by orders.customer_id. A customer can have many orders. Contact details are not extracted.                                       |
| public.service_types<br>(existing reference) | id — UUID<br>name — TEXT                                                                                                                                                                               | id          | Referenced by orders.service_type_id and the proposed service summary. Retain historical service references.                                |
| public.orders<br>(existing source)           | id — UUID<br>branch_id — UUID (NULL)<br>customer_id — UUID (NULL)<br>service_type_id — UUID (NULL)<br>created_at — TIMESTAMPTZ (NULL)<br>status — TEXT<br>weight_kg — NUMERIC<br>total_price — NUMERIC | id          | branch_id → branches.id<br>customer_id → customers.id<br>service_type_id → service_types.id<br>One order can have many payment entries.     |
| public.payments<br>(existing ledger)         | id — UUID<br>order_id — UUID<br>branch_id — UUID (NULL)<br>amount — NUMERIC<br>paid_at — TIMESTAMPTZ<br>source — TEXT                                                                                  | id          | order_id → orders.id<br>branch_id → branches.id<br>Service is resolved through the related order. Each collection is a separate ledger row. |

| Database Table                                               | Columns and Data Types                                                                                                                                                                                                                                                                                                                    | Primary Key                                             | Foreign Keys and Relationships                                                                                                                                    |
|--------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| public.expenses<br>(existing source)                         | id — UUID<br>branch_id — UUID (NULL)<br>amount — NUMERIC<br>expense_date — DATE<br>deleted_at — TIMESTAMPTZ (NULL)                                                                                                                                                                                                                        | id                                                      | branch_id → branches.id<br>Include only non-deleted expenses. A missing branch must be resolved before publication.                                               |
| analytics.<br>daily_branch_<br>summary<br>(proposed target)  | report_date — DATE<br>branch_id — UUID<br>order_count — BIGINT<br>released_order_count — BIGINT<br>unique_customer_count — BIGINT<br>total_weight_kg — NUMERIC(14,2)<br>order_value_php — NUMERIC(14,2)<br>payment_count — BIGINT<br>cash_received_php — NUMERIC(14,2)<br>expense_total_php — NUMERIC(14,2)<br>refreshed_at — TIMESTAMPTZ | Composite:<br>report_date,<br>branch_id                 | branch_id → public.branches.id<br>One row per date and branch. Merge order, payment, and expense aggregates using the union of their keys.                        |
| analytics.<br>daily_service_<br>summary<br>(proposed target) | report_date — DATE<br>branch_id — UUID<br>service_key — TEXT<br>service_type_id — UUID (NULL)<br>order_count — BIGINT<br>total_weight_kg — NUMERIC(14,2)<br>order_value_php — NUMERIC(14,2)<br>payment_count — BIGINT<br>cash_received_php — NUMERIC(14,2)<br>refreshed_at — TIMESTAMPTZ                                                  | Composite:<br>report_date,<br>branch_id,<br>service_key | branch_id → public.branches.id<br>service_type_id → public.service_types.id<br>One row per date, branch, and service key. Expenses are not assigned to a service. |

## Schema interpretation and ETL rules

Source versus ETL requirements. The source permits NULL branch IDs in orders, payments, and expenses, and NULL orders.created\_at. The proposed refresh requires resolved branches and dates before publication; it must not silently discard these rows or invent values. This reporting rule does not change the source constraints.

Staging: proposed pg\_temp.etl\_orders\_stage, pg\_temp.etl\_payments\_stage, and pg\_temp.etl\_expenses\_stage retain their source IDs and selected fields and add report\_date DATE. They exist only for one refresh and are dropped at transaction end.

Metric rules. Order counts, weight, and order value exclude cancelled orders and use the local creation date. Released-order counts describe current status within that creation-date group, not daily releases. Customer counts use distinct non-null IDs within a date and branch; do not sum them to infer distinct customers over a wider period.

Cash rules. cash\_received\_php is SUM(payments.amount), dated by paid\_at, with one count per ledger row. Later collections belong to the later date, not the order date. Do not also sum orders.amount\_paid. This proposed metric retains recorded collections after cancellation, which is not evidence of a refund. It is gross cash, not net income; net collections require an approved refund ledger. Legacy backfill timestamps are best-available estimates, not exact historical collection times.

Grouping and integrity. Branch summary keys include dates with only payments or expenses; service keys include dates with only payments. Missing measures are zero. service\_key is the service UUID as text, or unspecified for a null service ID. All target columns are required except service\_type\_id. Enforce unique composite keys, nonnegative totals, valid references, and service\_key consistency. Service totals for orders, weight, order value, payment count, and cash must reconcile to branch totals. Any discrepancy aborts the refresh.

Source-definition note. orders.status defaults to pending, but its supplied CHECK permits only received, on\_process, ready, released, and cancelled. The proposed ETL flags unsupported statuses; any default correction is a separate database change. orders.amount\_paid is nullable and is not the source of cash totals. No payments.voided\_at column is shown.

Schema basis and scope. The owner-supplied export is reference-only, not runnable migration SQL. It does not show the proposed analytics tables or verify triggers, access policies, or scheduled jobs. The other 15 public tables remain outside this reporting pipeline, including audit, inventory, loyalty, forecasts, and OTP records. Credentials and customer contact data are excluded.

## **6. Technology Justification**

### **PostgreSQL / Supabase**

The system already uses Supabase PostgreSQL for shared operational data. Keeping extraction and reporting storage in the same database fits the existing relational structure and avoids an additional data-transfer service. Keys support consistent relationships, and a transactional publication step can preserve the last successful summaries when a refresh fails. The job should run off-peak and its runtime should be monitored so analytical processing does not disrupt branch transactions.

### **Supabase Cron / pg\_cron**

This is the selected scheduler for the proposed daily SQL refresh. It provides an in-database trigger for the pipeline and job history for operational review, without introducing Apache Airflow or another scheduling platform. The configured scheduler timezone must be checked before enabling the 01:00 Asia/Manila schedule. An advisory lock in the refresh function prevents simultaneous runs from publishing conflicting results.

### **SQL**

SQL is selected for ingestion, validation, cleaning, transformation, and loading because the required data is relational. The same proposed function can apply date rules, validate references, aggregate separate transaction streams, and reconcile final totals. Independent aggregation is important: directly joining orders to multiple payments and expenses could multiply rows and overstate the results. Centralizing these rules also makes the proposed calculations easier to review and test.

### **Reporting Summary Tables**

These are the selected reporting destination, not an additional software product. The two proposed PostgreSQL tables store reusable measures at explicit date-and-branch and date-branch-and-service levels. They reduce repeated aggregation and separate order value from collected cash. Reports would display refreshed\_at and identify the completed-day cutoff; live order tracking would continue to use operational records. This separation supports consistent historical reporting without presenting scheduled summaries as real-time transaction data.